

資料結構

Data Structure

HW 1

組員 1

姓名： 黃聖傑

學號： 114310140

組員 2

姓名： 陳冠豪

學號： 114310125

分工表：

組員 1：程式架構、輸入資料處理、基礎運算功能、報告撰寫

組員 2：位元運算、比較功能、報告撰寫

HW01-Q1.

Implement a calculator application that evaluates mathematical expression.

Code

```
#include <iostream>
#include <cctype>
#include <string>
#include <sstream>
#include <limits>
#include <cstdlib>
#include <vector>
#include <algorithm>
#include <map>
#include <stdexcept>

using namespace std;

struct Node {
    char data;
    Node* next;
};

class Stack {
private:
    Node* top;
public:
    // 建立空的字元堆疊
    Stack() : top(nullptr) {}

    // 將字元推入堆疊
    void push(char ch) {
        Node* newNode = new Node;
        newNode->data = ch;
        newNode->next = top;
        top = newNode;
    }

    // 從堆疊取出最上層的字元
    char pop() {
        if (isEmpty()) return '\0';
        char ch = top->data;
        Node* temp = top;
```

```

        top = top->next;
        delete temp;
        return ch;
    }

    // 查看堆疊頂端的字元
    char peek() {
        return (top ? top->data : '\0');
    }

    // 判斷堆疊是否為空
    bool isEmpty() {
        return top == nullptr;
    }

    // 釋放堆疊中的所有節點
    ~Stack() {
        while (!isEmpty()) pop();
    }
};

struct NumNode {
    double data;
    NumNode* next;
};

class NumberStack {
private:
    NumNode* top;
public:
    // 建立空的數值堆疊
    NumberStack() : top(nullptr) {}

    // 將浮點數推入堆疊
    void push(double num) {
        NumNode* newNode = new NumNode;
        newNode->data = num;
        newNode->next = top;
    }
};

```

```

        top = newNode;
    }

    // 從堆疊取出最上層的數值
    double pop() {
        if (isEmpty()) return 0.0;
        double num = top->data;
        NumNode* temp = top;
        top = top->next;
        delete temp;
        return num;
    }

    // 查看堆疊頂端的數值
    double peek() {
        return (top ? top->data : 0.0);
    }

    // 判斷數值堆疊是否為空
    bool isEmpty() {
        return top == nullptr;
    }

    // 釋放數值堆疊中的所有節點
    ~NumberStack() {
        while (!isEmpty()) pop();
    }
};

// 去除字串前後空白
string trim(const string& input) {
    size_t start = 0;
    while (start < input.size() && isspace(static_cast<unsigned
char>(input[start]))) {
        ++start;
    }

    size_t end = input.size();

```

```

        while (end > start && isspace(static_cast<unsigned char>(input[end - 1])))
        {
            --end;
        }

        return input.substr(start, end - start);
    }

    // 判斷是否為二進位字面值
    bool isBinaryLiteral(const string& token) {
        if (token.size() < 3) return false;
        if (!(token[0] == '0' && (token[1] == 'b' || token[1] == 'B'))) return
false;
        for (size_t i = 2; i < token.size(); ++i) {
            if (token[i] != '0' && token[i] != '1') return false;
        }
        return token.size() > 2;
    }

    // 將二進位或十進位字串轉成整數
    long long parseBitInteger(const string& token) {
        if (isBinaryLiteral(token)) {
            long long value = 0;
            for (size_t i = 2; i < token.size(); ++i) {
                value = (value << 1) | (token[i] == '1' ? 1 : 0);
            }
            return value;
        }
        return stoll(token);
    }

    // 判斷字串是否為純二進位數字
    bool isBinaryDigits(const string& token) {
        if (token.empty()) return false;
        for (char ch : token) {
            if (ch != '0' && ch != '1') {
                return false;
            }
        }
    }

```

```

    }
    return true;
}

// 將純二進位字串轉成整數
long long parseBinaryDigits(const string& token) {
    long long value = 0;
    for (char ch : token) {
        value = (value << 1) | (ch == '1' ? 1 : 0);
    }
    return value;
}

// 根據模式把 token 轉換成數值
double parseNumericToken(const string& token, bool bitwiseMode) {
    if (isBinaryLiteral(token)) {
        return static_cast<double>(parseBitInteger(token));
    }
    if (bitwiseMode && isBinaryDigits(token)) {
        return static_cast<double>(parseBinaryDigits(token));
    }
    return stod(token);
}

// 取得位元 token 的位寬，若不是位元字面值則回傳 0
int bitWidthOfToken(const string& token, bool bitwiseMode) {
    if (isBinaryLiteral(token)) {
        return static_cast<int>(token.size() - 2);
    }
    if (bitwiseMode && isBinaryDigits(token)) {
        return static_cast<int>(token.size());
    }
    return 0;
}

unsigned long long maskForWidth(int width) {
    if (width <= 0) {
        return ~0ULL;
    }
}

```



```

    }
    if (width >= 64) {
        return ~0ULL;
    }
    return (1ULL << width) - 1ULL;
}

// 判斷字元是否為運算子或括號
bool isOperatorChar(char ch) {
    return ch == '+' || ch == '-' || ch == '*' || ch == '/' || ch == '%' ||
           ch == '^' || ch == '(' || ch == ')' || ch == '!' || ch == '<' ||
           ch == '>' || ch == '&' || ch == '|' || ch == '~' || ch == '=';
}

// 將關鍵字或符號對應成內部代表的運算子
char encodeOperator(const string& token) {
    if (token == "<=") return 'L';
    if (token == ">=") return 'G';
    if (token == "==") return 'E';
    if (token == "!=") return 'N';
    if (token == "&&") return 'A';
    if (token == "||") return 'O';
    if (token == "<<") return 'S';
    if (token == ">>") return 'R';
    if (token == "~") return '~';
    if (token == "!") return '!';
    if (token == "<") return '<';
    if (token == ">") return '>';
    if (token == "&") return '&';
    if (token == "|") return '|';
    if (token == "^") return '^';
    if (token == "+") return '+';
    if (token == "-") return '-';
    if (token == "*") return '*';
    if (token == "/") return '/';
    if (token == "%") return '%';
    if (token == "(") return '(';
    if (token == ")") return ')';
}

```

```

    if (token == "=") return '=';
    if (token == "u-" || token == "M") return 'M';
    if (token == "AND" || token == "and") return 'A';
    if (token == "OR" || token == "or") return 'O';
    if (token == "XOR" || token == "xor") return 'X';
    if (token == "NOT" || token == "not") return '!';
    return '\0';
}

// 取得 token 對應的内部運算子代碼
char resolveTokenOperator(const string& token) {
    char op = encodeOperator(token);
    if (op != '\0') return op;
    if (token.size() == 1) {
        char ch = token[0];
        if (ch == 'L' || ch == 'G' || ch == 'E' || ch == 'N' || ch == 'A' || ch
== 'O' || ch == 'X' || ch == 'S' || ch == 'R' || ch == 'M') {
            return ch;
        }
    }
    return '\0';
}

// 設定各種運算子的優先權
int precedence(char op) {
    switch (op) {
        case 'M':
        case '~':
        case '!':
            return 50;
        case 'S':
        case 'R':
            return 45;
        case '^':
            return 40;
        case '*':
        case '/':
        case '%':

```

```

        return 35;
    case '+':
    case '-':
        return 30;
    case '<':
    case '>':
    case 'L':
    case 'G':
        return 25;
    case 'E':
    case 'N':
        return 23;
    case '&':
        return 20;
    case 'A':
        return 18;
    case 'X':
        return 16;
    case 'O':
        return 14;
    case '=':
        return 1;
    default:
        return -1;
}
}

// 判斷是否為有效的運算子
bool isOperator(char ch) {
    return precedence(ch) > 0;
}

// 判斷數值是否代表真值
bool isTruthy(double value) {
    return value != 0.0;
}

// 將整數轉成二進位字串，若指定位寬則補足前導 0

```

```

string toBinaryString(long long value, int width = 0) {
    if (width <= 0) {
        if (value == 0) return "0";
        string result;
        bool negative = value < 0;
        unsigned long long magnitude = negative ? static_cast<unsigned long long>(-(value + 1)) + 1 : static_cast<unsigned long long>(value);

        while (magnitude > 0) {
            result.push_back((magnitude & 1ULL) ? '1' : '0');
            magnitude >>= 1;
        }

        if (negative) {
            result.push_back('-');
        }

        reverse(result.begin(), result.end());
        return result;
    }

    unsigned long long magnitude = static_cast<unsigned long long>(value) &
maskForWidth(width);
    string result(width, '0');
    for (int i = width - 1; i >= 0; --i) {
        result[i] = (magnitude & 1ULL) ? '1' : '0';
        magnitude >>= 1;
    }
    return result;
}

// 格式化位元運算結果，保留輸入時的位元數會補足高位元 0
string formatBitResult(long long result, int width = 0) {
    return to_string(result) + " (binary: " + toBinaryString(result, width) +
    ")";
}

enum class ValueKind {

```

```

    Number,
    Boolean
};

struct VariableValue {
    double value;
    ValueKind kind;
    int bitWidth;
};

struct EvalValue {
    ValueKind kind;
    double number;
    string variableName;
    bool isVariable;
    int bitWidth;
};

map<string, VariableValue> variables;

string formatLogicalValue(double value);

// 判斷是否為布林值
bool isBooleanLiteral(const string& token) {
    return token == "true" || token == "false";
}

// 判斷是否為合法的識別字
bool isIdentifierToken(const string& token) {
    if (token.empty()) return false;
    if (isdigit(static_cast<unsigned char>(token[0]))) return false;
    if (isBooleanLiteral(token)) return false;
    if (encodeOperator(token) != '\0') return false;
    for (char ch : token) {
        if (!isalnum(static_cast<unsigned char>(ch)) && ch != '_') {
            return false;
        }
    }
}

```

```

        return true;
    }

    // 建立數值型的評估結果
    EvalValue makeNumber(double value, int bitWidth = 0) {
        return {ValueKind::Number, value, "", false, bitWidth};
    }

    // 建立布林型的評估結果
    EvalValue makeBoolean(double value) {
        return {ValueKind::Boolean, value, "", false, 0};
    }

    // 建立變數型的評估結果
    EvalValue makeVariable(const string& name) {
        return {ValueKind::Number, 0.0, name, true, 0};
    }

    // 將變數或常數轉換成數值
    double resolveNumericValue(const EvalValue& value, const map<string,
    VariableValue>& variables) {
        if (value.isVariable) {
            auto it = variables.find(value.variableName);
            if (it == variables.end()) {
                cout << "Error: Undefined variable '" << value.variableName << "'!"
                << endl;
                return 0.0;
            }
            return it->second.value;
        }
        return value.number;
    }

    // 取得評估值目前的類型
    ValueKind resolveValueKind(const EvalValue& value, const map<string,
    VariableValue>& variables) {
        if (value.isVariable) {
            auto it = variables.find(value.variableName);

```

```

        if (it == variables.end()) {
            return ValueKind::Number;
        }
        return it->second.kind;
    }
    return value.kind;
}

// 取得評估值目前的位元數
int resolveBitWidth(const EvalValue& value, const map<string, VariableValue>&
variables) {
    if (value.isVariable) {
        auto it = variables.find(value.variableName);
        if (it == variables.end()) {
            return value.bitWidth;
        }
        return it->second.bitWidth;
    }
    return value.bitWidth;
}

// 將評估值格式化為輸出字串
string formatValue(const EvalValue& value, const map<string, VariableValue>&
variables) {
    ValueKind kind = resolveValueKind(value, variables);
    if (kind == ValueKind::Boolean) {
        return formatLogicalValue(resolveNumericValue(value, variables));
    }
    return to_string(resolveNumericValue(value, variables));
}

// 執行二元算術運算
double applyBinaryArithmetic(double a, double b, char op) {
    switch (op) {
        case '+': return a + b;
        case '-': return a - b;
        case '*': return a * b;
        case '/':
    }

```

```

        if (b == 0) {
            cout << "Error: Division by zero!" << endl;
            return 0.0;
        }
        return a / b;
    case '%':
        return static_cast<long long>(a) % static_cast<long long>(b);
    case '^': {
        double result = 1.0;
        for (int i = 0; i < static_cast<int>(b); ++i) {
            result *= a;
        }
        return result;
    }
    default:
        return 0.0;
}
}

```

// 執行比較與邏輯二元運算

```

double applyLogicalOperation(double a, double b, char op) {
    switch (op) {
        case '<': return a < b ? 1.0 : 0.0;
        case '>': return a > b ? 1.0 : 0.0;
        case 'L': return a <= b ? 1.0 : 0.0;
        case 'G': return a >= b ? 1.0 : 0.0;
        case 'E': return a == b ? 1.0 : 0.0;
        case 'N': return a != b ? 1.0 : 0.0;
        case 'A': return (isTruthy(a) && isTruthy(b)) ? 1.0 : 0.0;
        case 'O': return (isTruthy(a) || isTruthy(b)) ? 1.0 : 0.0;
        case 'X': return (isTruthy(a) != isTruthy(b)) ? 1.0 : 0.0;
        default:
            return 0.0;
    }
}

```

// 將邏輯結果格式化為 T / F

```

string formatLogicalValue(double value) {

```



```

    return isTruthy(value) ? "T" : "F";
}

// 執行位元運算
long long applyBitwiseOp(long long a, long long b, char op, int width = 0) {
    if (width <= 0) {
        switch (op) {
            case '|': return a | b;
            case '&': return a & b;
            case '^': return a ^ b;
            case 'S': return a << b;
            case 'R': return a >> b;
            default:
                return 0;
        }
    }

    unsigned long long ua = static_cast<unsigned long long>(a) &
maskForWidth(width);
    unsigned long long ub = static_cast<unsigned long long>(b) &
maskForWidth(width);
    unsigned long long result = 0;

    switch (op) {
        case '|': result = ua | ub; break;
        case '&': result = ua & ub; break;
        case '^': result = ua ^ ub; break;
        case 'S': result = (ua << static_cast<unsigned long long>(b)) &
maskForWidth(width); break;
        case 'R': result = ua >> static_cast<unsigned long long>(b); break;
        default:
            return 0;
    }

    return static_cast<long long>(result);
}

// 執行一元邏輯運算

```

```

double applyUnaryLogic(char op, double operand) {
    if (op == '!') {
        return isTruthy(operand) ? 0.0 : 1.0;
    }
    return 0.0;
}

// 印出目前步驟
void printStep(int& step, const string& label) {
    cout << "Step " << step++ << ": " << label << endl;
}

// 判斷某個 token 是否為一元運算子
bool isUnaryToken(const string& token, const vector<string>& tokens, size_t
index);

// 將 infix expression 切成 token
bool tokenize(const string& infix, vector<string>& tokens) {
    size_t i = 0;
    while (i < infix.size()) {
        if (isspace(static_cast<unsigned char>(infix[i]))) {
            ++i;
            continue;
        }

        if (isdigit(static_cast<unsigned char>(infix[i])) || infix[i] == '.' ||
            (infix[i] == '0' && i + 1 < infix.size() && (infix[i + 1] == 'b' ||
infix[i + 1] == 'B')))) {
            size_t start = i;
            if (infix[i] == '0' && i + 1 < infix.size() && (infix[i + 1] == 'b'
|| infix[i + 1] == 'B')) {
                i += 2;
                while (i < infix.size() && (infix[i] == '0' || infix[i] == '1'))
                {
                    ++i;
                }
            } else {

```

```

        while (i < infix.size() && (isdigit(static_cast<unsigned
char>(infix[i])) || infix[i] == '.')) {
            ++i;
        }
    }
    tokens.push_back(infix.substr(start, i - start));
    continue;
}

if (isalpha(static_cast<unsigned char>(infix[i]))) {
    size_t start = i;
    while (i < infix.size() && isalpha(static_cast<unsigned
char>(infix[i]))) {
        ++i;
    }
    tokens.push_back(infix.substr(start, i - start));
    continue;
}

if (infix.compare(i, 2, "<=") == 0 || infix.compare(i, 2, ">=") == 0 ||
    infix.compare(i, 2, "==") == 0 || infix.compare(i, 2, "!=") == 0 ||
    infix.compare(i, 2, "&&") == 0 || infix.compare(i, 2, "||") == 0 ||
    infix.compare(i, 2, "<<") == 0 || infix.compare(i, 2, ">>") == 0) {
    tokens.push_back(infix.substr(i, 2));
    i += 2;
    continue;
}

if (isOperatorChar(infix[i])) {
    if (infix[i] == '-' && isUnaryToken("-", tokens, tokens.size())) {
        tokens.push_back("u-");
    } else {
        tokens.push_back(string(1, infix[i]));
    }
    ++i;
    continue;
}

```

```

        cout << "Error: Invalid character '" << infix[i] << "' at position " <<
i << "!" << endl;
        return false;
    }

    return true;
}

// 將 infix 轉成 postfix
void convertToPostfix(const vector<string>& tokens, string& postfix) {
    Stack operators;
    postfix.clear();

    for (const string& token : tokens) {
        char op = encodeOperator(token);
        if (op == '\\0') {
            if (!postfix.empty()) postfix += ' ';
            postfix += token;
            postfix += ' ';
            continue;
        }

        if (op == '(') {
            operators.push(op);
            continue;
        }

        if (op == ')') {
            while (!operators.isEmpty() && operators.peek() != '(') {
                if (!postfix.empty()) postfix += ' ';
                postfix += operators.pop();
                postfix += ' ';
            }
            if (!operators.isEmpty() && operators.peek() == '(') {
                operators.pop();
            }
            continue;
        }
    }
}

```

```

        while (!operators.isEmpty() && operators.peek() != '(' &&
precedence(operators.peek()) >= precedence(op)) {
            if (!postfix.empty()) postfix += ' ';
            postfix += operators.pop();
            postfix += ' ';
        }
        operators.push(op);
    }

    while (!operators.isEmpty()) {
        if (!postfix.empty()) postfix += ' ';
        postfix += operators.pop();
        postfix += ' ';
    }

    while (!postfix.empty() && postfix.back() == ' ') postfix.pop_back();
}

// 判斷某個 token 是否為 1 元運算子
bool isUnaryToken(const string& token, const vector<string>& tokens, size_t
index) {
    if (token != "!" && token != "~" && token != "-") return false;
    if (index == 0) return true;
    const string& prev = tokens[index - 1];
    char prevOp = encodeOperator(prev);
    return prevOp == '(' || isOperator(prevOp);
}

// 依照 postfix 計算並輸出每一步驟
void evaluatePostfix(const string& postfix, bool bitwiseMode, int& step) {
    vector<EvalValue> values;
    stringstream ss(postfix);
    string token;

    auto popValue = [&]() -> EvalValue {
        if (values.empty()) {

```

```

        cout << "Error: Invalid postfix expression (insufficient operands)!"
<< endl;

        throw runtime_error("insufficient operands");
    }
    EvalValue value = values.back();
    values.pop_back();
    return value;
};

auto pushValue = [&](const EvalValue& value) {
    values.push_back(value);
};

while (ss >> token) {
    char op = resolveTokenOperator(token);

    if (bitwiseMode) {
        if (op == 'A') op = '&';
        else if (op == 'O') op = '|';
        else if (op == 'X') op = '^';
        else if (token == "NOT" || token == "not") op = '~';
    }

    if (op == '\\0') {
        EvalValue value;
        if (isBooleanLiteral(token)) {
            value = makeBoolean(token == "true" ? 1.0 : 0.0);
            printStep(step, "Push " + string(token == "true" ? "true" :
"false"));
        } else if (isIdentifierToken(token)) {
            value = makeVariable(token);
            printStep(step, "Load " + token);
        } else {
            try {
                int bitWidth = bitWidthOfToken(token, bitwiseMode);
                value = makeNumber(parseNumericToken(token, bitwiseMode),
bitWidth);
            } catch (...) {

```

```

        cout << "Error: Invalid token '" << token << "'" << endl;
        return;
    }
    printStep(step, "Push " + to_string(value.number));
}
pushValue(value);
continue;
}

if (op == '=') {
    EvalValue rhs = popValue();
    EvalValue lhs = popValue();
    if (!lhs.isVariable) {
        cout << "Error: Assignment target must be a variable!" << endl;
        return;
    }
    double assignedValue = resolveNumericValue(rhs, variables);
    ValueKind assignedKind = resolveValueKind(rhs, variables);
    int assignedWidth = resolveBitWidth(rhs, variables);
    variables[lhs.variableName] = {assignedValue, assignedKind,
assignedWidth};
    printStep(step, "Assign " + lhs.variableName + " = " + (assignedKind
== ValueKind::Boolean ? formatLogicalValue(assignedValue) :
to_string(assignedValue)));
    if (assignedKind == ValueKind::Boolean) {
        pushValue(makeBoolean(assignedValue));
    } else {
        pushValue(makeNumber(assignedValue, assignedWidth));
    }
    continue;
}

if (op == '!' || op == '~') {
    EvalValue operand = popValue();
    if (op == '!') {
        double result = applyUnaryLogic('!',
resolveNumericValue(operand, variables));

```

```

        printStep(step, "! " + formatValue(operand, variables) + " = " +
formatLogicalValue(result));
        pushValue(makeBoolean(result));
    } else {
        long long operandValue = static_cast<long
long>(resolveNumericValue(operand, variables));
        int operandWidth = resolveBitWidth(operand, variables);
        long long result = operandValue;
        if (operandWidth > 0) {
            unsigned long long ua = static_cast<unsigned long
long>(operandValue) & maskForWidth(operandWidth);
            result = static_cast<long long>((~ua) &
maskForWidth(operandWidth));
        } else {
            result = ~operandValue;
        }
        printStep(step, "~ " + to_string(static_cast<long
long>(resolveNumericValue(operand, variables))) + " = " +
formatBitResult(result, operandWidth));
        pushValue(makeNumber(static_cast<double>(result),
operandWidth));
    }
    continue;
}

if (op == 'M') {
    EvalValue operand = popValue();
    double operandValue = resolveNumericValue(operand, variables);
    int operandWidth = resolveBitWidth(operand, variables);
    double result = -operandValue;
    printStep(step, "- " + formatValue(operand, variables) + " = " +
to_string(result));
    pushValue(makeNumber(result, operandWidth));
    continue;
}

if (bitwiseMode) {
    if (op == '|' || op == '&' || op == '^' || op == 'S' || op == 'R') {

```



```

        EvalValue rhs;
        EvalValue lhs;
        if (op == 'S' || op == 'R') {
            if (values.size() == 1) {
                rhs = makeNumber(1.0);
                lhs = popValue();
            } else {
                rhs = popValue();
                lhs = popValue();
            }
        } else {
            rhs = popValue();
            lhs = popValue();
        }
        long long a = static_cast<long long>(resolveNumericValue(lhs,
variables));
        long long b = static_cast<long long>(resolveNumericValue(rhs,
variables));

        int lhsWidth = resolveBitWidth(lhs, variables);
        int rhsWidth = resolveBitWidth(rhs, variables);
        int width = lhsWidth > 0 ? lhsWidth : rhsWidth;
        long long result = applyBitwiseOp(a, b, op, width);
        printStep(step, to_string(a) + " " + string(1, op) + " " +
to_string(b) + " = " + formatBitResult(result, width));
        pushValue(makeNumber(static_cast<double>(result), width));
        continue;
    }
    cout << "Error: Invalid operator in bitwise mode: " << token <<
endl;

    return;
}

EvalValue rhs = popValue();
EvalValue lhs = popValue();

    if (op == '+' || op == '-' || op == '*' || op == '/' || op == '%' || op
== '^') {
        double a = resolveNumericValue(lhs, variables);

```

```

        double b = resolveNumericValue(rhs, variables);
        double result = applyBinaryArithmetic(a, b, op);
        printStep(step, to_string(a) + " " + string(1, op) + " " +
to_string(b) + " = " + to_string(result));
        pushValue(makeNumber(result));
        continue;
    }

    double a = resolveNumericValue(lhs, variables);
    double b = resolveNumericValue(rhs, variables);
    double result = applyLogicalOperation(a, b, op);
    printStep(step, to_string(a) + " " + string(1, op) + " " + to_string(b)
+ " = " + formatLogicalValue(result));
    pushValue(makeBoolean(result));
}

if (values.empty()) {
    cout << "Error: Invalid postfix expression (empty)!" << endl;
    return;
}

EvalValue result = values.back();
values.pop_back();
if (!values.empty()) {
    cout << "Error: Invalid postfix expression (too many operands)!" <<
endl;
    return;
}

if (bitwiseMode) {
    cout << "Result: " << formatBitResult(static_cast<long
long>(resolveNumericValue(result, variables)), resolveBitWidth(result,
variables)) << endl;
} else if (result.kind == ValueKind::Boolean) {
    cout << "Result: " << formatLogicalValue(resolveNumericValue(result,
variables)) << endl;
} else {
    cout << "Result: " << resolveNumericValue(result, variables) << endl;
}

```

```

    }
}

// 處理使用者選擇的模式，包含輸入、轉換、判斷與輸出
bool processMode(int mode) {
    string infix;
    cout << "\nMode " << mode << " - enter expression (type /0 to return to
menu): ";
    getline(cin, infix);
    infix = trim(infix);

    if (infix == "/0") {
        return true;
    }

    if (infix.empty()) {
        cout << "Error: Empty expression!" << endl;
        return false;
    }

    vector<string> tokens;
    if (!tokenize(infix, tokens)) {
        return false;
    }

    if (tokens.empty()) {
        cout << "Error: Empty expression!" << endl;
        return false;
    }

    string postfix;
    convertToPostfix(tokens, postfix);

    if (postfix.empty()) {
        cout << "Error: Invalid expression!" << endl;
        return false;
    }
}

```

```

    cout << "Postfix expression: " << postfix << endl;
    cout << "\n--- Evaluation Steps ---" << endl;
    int step = 1;
    evaluatePostfix(postfix, mode == 2, step);
    return false;
}

// 程式主迴圈，顯示選單並根據使用者選擇執行對應模式
int main() {
    while (true) {
        //功能選擇
        cout << "\n==== Calculator Menu =====> endl;
        cout << "1. Basic arithmetic + logical judgment" << endl;
        cout << "2. Bitwise operations" << endl;
        cout << "3. Exit" << endl;
        cout << "Choose an option: ";

        string choiceLine;
        getline(cin, choiceLine);
        choiceLine = trim(choiceLine);

        if (choiceLine == "/0") {
            cout << "Returning to menu..." << endl;
            continue;
        }

        int choice = 0;
        try {
            choice = stoi(choiceLine);
        } catch (...) {
            cout << "Error: Invalid menu option!" << endl;
            continue;
        }

        if (choice == 1) {
            while (!processMode(1)) {

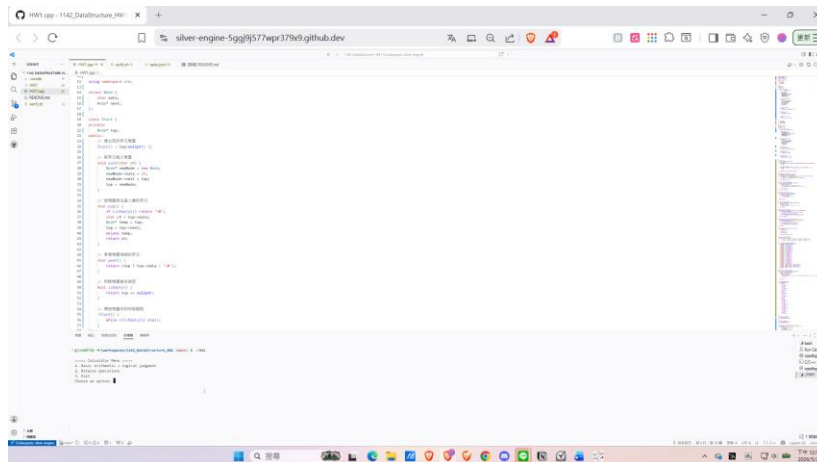
            }
        }
    }
}

```

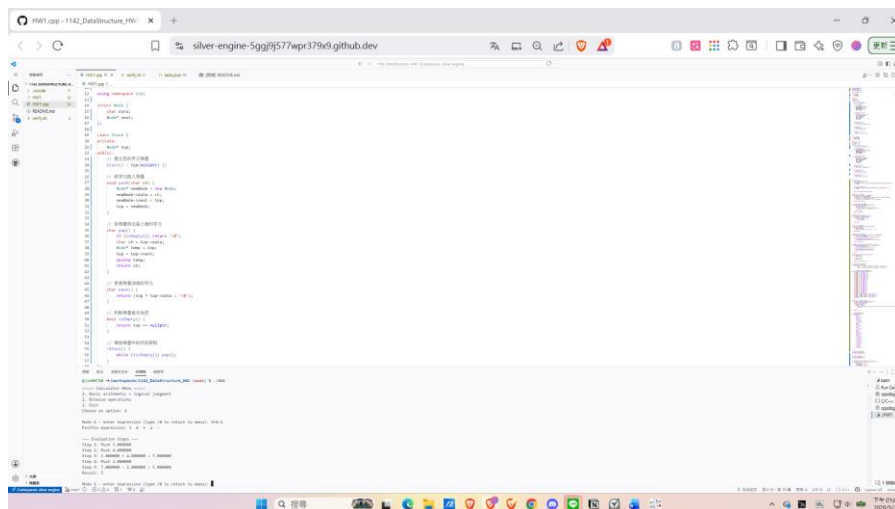
```
    } else if (choice == 2) {  
        while (!processMode(2)) {  
  
        }  
    } else if (choice == 3) {  
        cout << "Exiting program." << endl;  
        break;  
    } else {  
        cout << "Error: Invalid menu option!" << endl;  
    }  
}  
  
return 0;  
}
```

Result

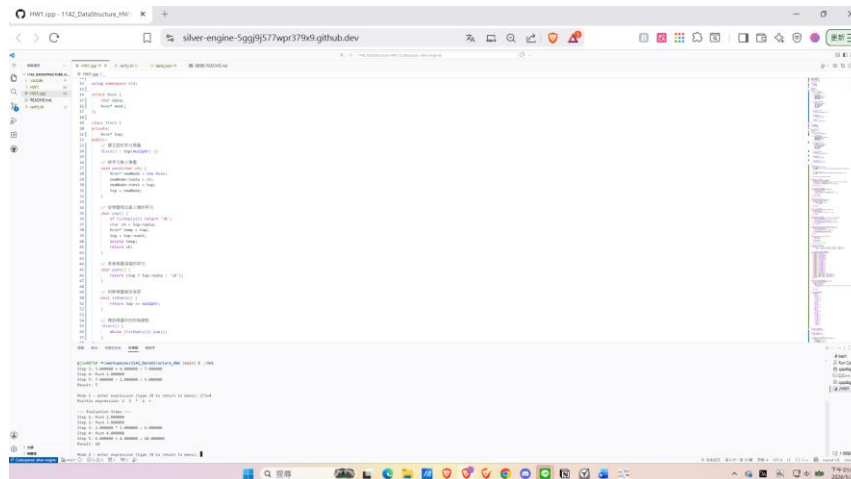
功能選擇(數字運算 or 位元運算)



測試 1



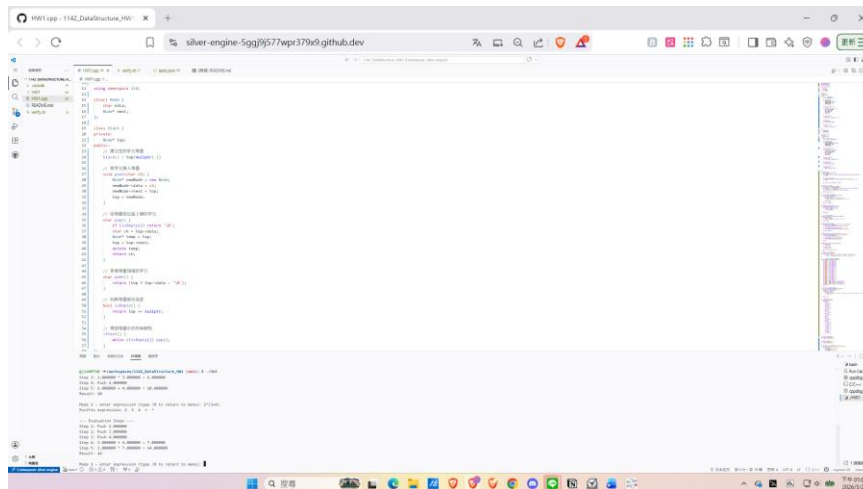
測試 2



```
Mode 1 - enter expression (type /0 to return to menu): 2*3+4
Postfix expression: 2 3 * 4 +

--- Evaluation Steps ---
Step 1: Push 2.000000
Step 2: Push 3.000000
Step 3: 2.000000 * 3.000000 = 6.000000
Step 4: Push 4.000000
Step 5: 6.000000 + 4.000000 = 10.000000
Result: 10
```

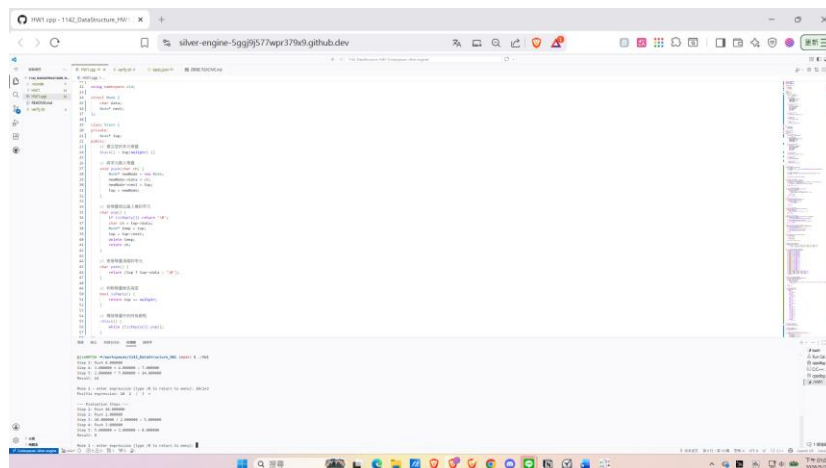
測試 3



```
Mode 1 - enter expression (type /0 to return to menu): 2*(3+4)
Postfix expression: 2 3 4 + *

--- Evaluation Steps ---
Step 1: Push 2.000000
Step 2: Push 3.000000
Step 3: Push 4.000000
Step 4: 3.000000 + 4.000000 = 7.000000
Step 5: 2.000000 * 7.000000 = 14.000000
Result: 14
```

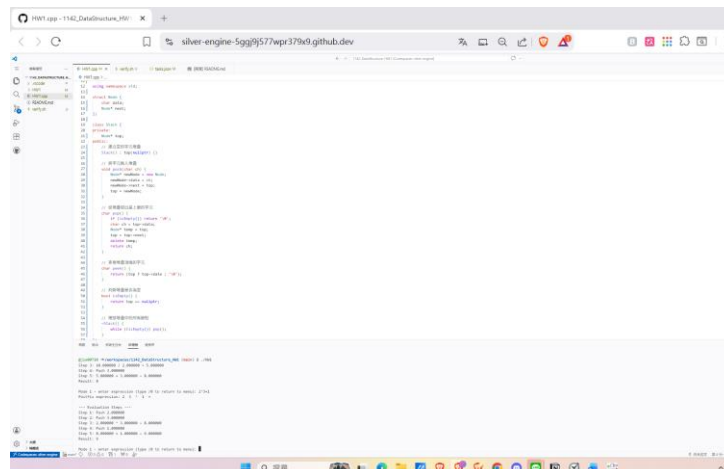
測試 4



```
Mode 1 - enter expression (type /0 to return to menu): 10/2+3
Postfix expression: 10 2 / 3 +

--- Evaluation Steps ---
Step 1: Push 10.000000
Step 2: Push 2.000000
Step 3: 10.000000 / 2.000000 = 5.000000
Step 4: Push 3.000000
Step 5: 5.000000 + 3.000000 = 8.000000
Result: 8
```

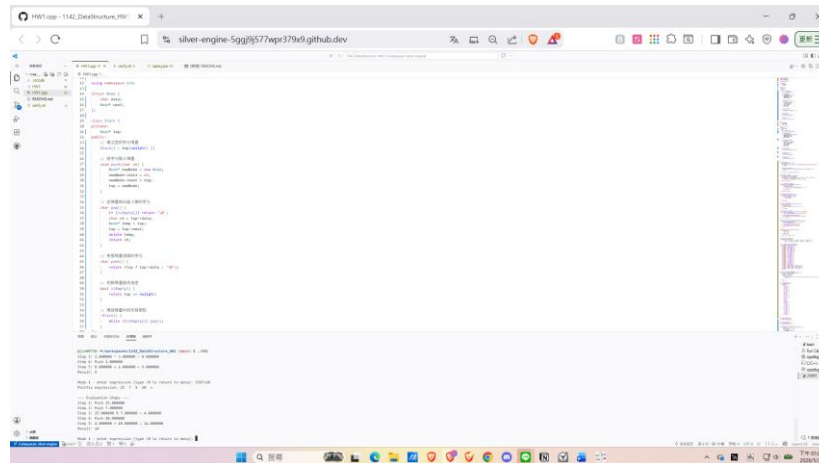
測試 5



```
Mode 1 - enter expression (type /0 to return to menu): 2^3+1
Postfix expression: 2 3 ^ 1 +

--- Evaluation Steps ---
Step 1: Push 2.000000
Step 2: Push 3.000000
Step 3: 2.000000 ^ 3.000000 = 8.000000
Step 4: Push 1.000000
Step 5: 8.000000 + 1.000000 = 9.000000
Result: 9
```

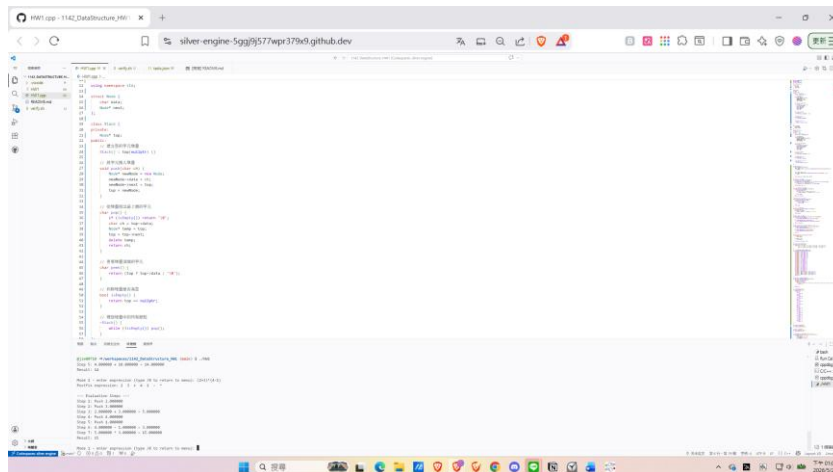

測試 6



```
Mode 1 - enter expression (type /0 to return to menu): 25%7+10
Postfix expression: 25 7 % 10 +

--- Evaluation Steps ---
Step 1: Push 25.000000
Step 2: Push 7.000000
Step 3: 25.000000 % 7.000000 = 4.000000
Step 4: Push 10.000000
Step 5: 4.000000 + 10.000000 = 14.000000
Result: 14
```

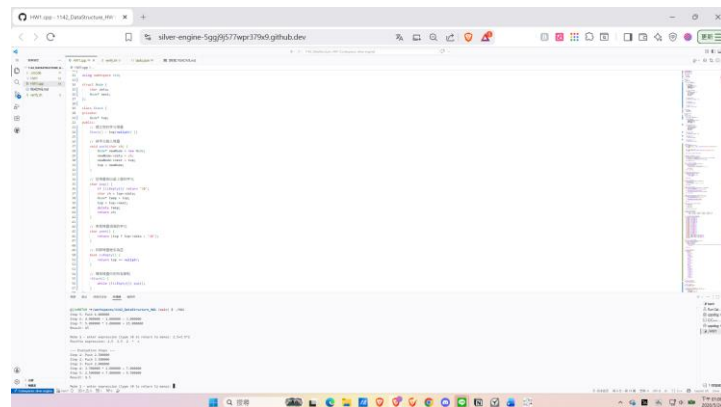
測試 7



```
Mode 1 - enter expression (type /0 to return to menu): (2+3)*(4-1)
Postfix expression: 2 3 + 4 1 - *

--- Evaluation Steps ---
Step 1: Push 2.000000
Step 2: Push 3.000000
Step 3: 2.000000 + 3.000000 = 5.000000
Step 4: Push 4.000000
Step 5: Push 1.000000
Step 6: 4.000000 - 1.000000 = 3.000000
Step 7: 5.000000 * 3.000000 = 15.000000
Result: 15
```

測試 8



```
Mode 1 - enter expression (type /0 to return to menu): 2.5+3.5*2
Postfix expression: 2.5 3.5 2 * +

--- Evaluation Steps ---
Step 1: Push 2.500000
Step 2: Push 3.500000
Step 3: Push 2.000000
Step 4: 3.500000 * 2.000000 = 7.000000
Step 5: 2.500000 + 7.000000 = 9.500000
Result: 9.5
```

Appendix

設計方式：

1. 資料結構：Singly Linked List, stack

使用兩個 stack 用於儲存字元以便後續 infix⇒postfix 轉換，及 Number stack 用於儲存數值，再利用堆疊中的元素建立 std:vector 執行計算

2. Infix 轉換為 Postfix

遇到運算元：直接輸出

遇到左括號：推入 stack

遇到右括號：將堆疊彈出至左括號，左括號丟棄

遇到運算子：與頂端運算子比較優先權，若頂端運算子較高，則將頂端運算子彈出
當全部 Token 掃描完畢，將堆疊全部輸出

3. 時間複雜度分析

掃描： $O(n)$

中綴轉後綴： $O(n)$

讀取後綴並運算： $O(n)$

總結： $O(n)$

4. 遇到的挑戰與解決方式

在開發這個計算機系統的過程中，遇到了不少實作上的困難。其中最大的挑戰之一，是處理中序運算式（Infix Expression）轉換成後序運算式（Postfix Expression）時的運算子優先權問題。由於程式同時支援算術運算、邏輯判斷、比較運算以及位元運算，因此必須仔細規劃各種運算子的優先順序，才能確保最終計算結果正確。為了解決這個問題，程式中設計了 precedence 函式來定義不同運算子的優先權。另外，一元負號（Unary Minus）與一般減法運算子的區分也是一項困難。因為符號「-」有時代表負數，

有時則代表減法，因此需要依照不同情況進行判斷。為了解決這個問題，程式中額外使用「u-」作為一元負號的內部表示方式，並在 tokenization 與 postfix 轉換階段進行判別，讓系統能正確辨識負數。此外，token parsing 的設計也相對複雜，因為程式需要辨識多種不同型態的輸入，包括十進位數字、二進位數字、變數名稱，以及像是「<=」、「>=」、「&&」、「||」等多字元運算子。因此程式採用了逐字元分析的方式，並加入多字元運算子的判斷機制，以避免 token 切割錯誤。在 postfix 計算過程中，stack 的管理也是一個重要問題。如果使用者輸入不合法的運算式，或是運算元不足，就可能發生 stack underflow 的情況。因此程式中加入了額外的錯誤檢查機制，用來判斷 postfix expression 是否合法，避免程式在執行時直接崩潰。最後，由於整合了算術、邏輯與位元運算，因此整體程式架構相對複雜。為了讓不同模式都能正確運作，程式中針對不同運算模式設計了獨立的運算邏輯，提升系統的正确性。

實作過程：

小組使用 GitHub Copilot 製作運算符號評估值(優先順序)的轉換，及位元運算的操作並保留先前作業 stack 結構，加入基本算數運算元，此部分為組員 1 完成，先利用課程 Lab 程式將 Infix 輸入利用 stack 轉換成 postfix，再依序讀取表達式裡的每個位元，遇到 2 元運算子時，抓取前兩位運算元執行後，將結果推入 stack，若遇到 1 元運算子(如 NOT 則將 stack 裡的 top 取出 1,0 互換後再放入 stack)，及輸出操作過程，最後輸出 stack 計算後留下的最後一個元素，使用 AI 補齊所需功能後，進行測試及部分修正，修正位元運算後輸出二進制結果會以最小表示位元輸出，例如 0101 會輸出 101，加入判斷位元數的功能，若計算結果不足位元數，則在前端補 0 至達輸入最高位元(此部分為組員 2 修正)，小組成員共同測試程式操作及完成報告與程式註解。

AI 命令：

請保留程式碼中的 stack 結構與數字運算不變，加入功能選擇清單，功能如下

- 1.基本代數運算(即程式碼中已完成段落)
- 2.位元操作(兼容以下操作(&and, |or, ^xor, ~not, <<left shift, >>right shift))

舉例：1100>>得到 0110

1100&1010 得到 1000

3.離開程式

請在以上任一功能，輸入/0 回到功能選單

並在我的程式中加入位元操作的功能，並使得功能 1 兼容邏輯判斷

邏輯判斷結果以 T/F 呈現

須判斷以下比較運算元(<, >, ==, !=, <=, >=)、及以下連接詞(AND, OR, NOT, XOR)

針對 AI 生成內容之修改與改進討論：

在本次專題開發過程中，我們是先提供基礎功能需求與部分運算邏輯給 AI 作為參考，再由小組成員依照作業要求進一步修改與擴充程式內容。例如原本程式僅能進行基本數學運算，我們後續自行加入了位元運算功能，包含 AND、OR、XOR、NOT、左移與右移等操作，並保留原本以 stack 為核心的資料結構。此外，我們也新增了功能選單與 /0 返回主選單的設計，提升操作便利性。在邏輯判斷部分，也擴充了比較運算子與邏輯連接詞的功能，並將結果以 T/F 的形式輸出。同時，我們也針對 AI 產生的內容進行測試與除錯，修正部分位元運算輸出格式不完整的問題，例如加入位元數判斷與前導 0 補齊功能，使輸出結果更符合題目需求。